GUÍA DE IMPLEMENTACIÓN

Demo local de agente con Ollama, Harness y Opik

Construcción paso a paso, desde una terminal vacía hasta un agente observable y limitado por software.

Node.js	TypeScript	Ollama	Harness	Opik

Objetivo del documento

Tener un recorrido pequeño y verificable para construir el demo final de la plática universitaria: el mismo LLM pasa de responder texto a usar herramientas, entrar en un agent loop, obedecer controles externos y dejar un trace visible en Opik.

> **Regla del proyecto**
>
> No vamos a construir “una plataforma de agentes”. Vamos a construir el ejemplo mínimo que demuestre con claridad LLM → Tools → Agent → Harness → Observability.

Versión de trabajo · 25 de septiembre de 2026

Cómo usar esta guía

Construye una fase, prueba el checkpoint y solo entonces continúa.

La guía evita deliberadamente darte la implementación completa. La idea es que tú construyas cada pieza y entiendas qué responsabilidad tiene antes de añadir la siguiente.

Resultado final



Criterios de éxito

- ☐ Cada tool tiene una responsabilidad obvia y pequeña.
- ☐ La seguridad no depende de que el prompt diga “pórtate bien”.
- ☐ El demo funciona sin Internet una vez descargados modelo y dependencias.
- ☐ Un trace de Opik permite seguir la ejecución completa.

Alcance congelado

SÍ CONSTRUIMOS	NO CONSTRUIMOS
Terminal interactiva	UI web
Ollama local	Cloud LLM
3 tools	MCP
Agent loop hecho a mano	LangChain / CrewAI / LangGraph
Harness mínimo	Autorización compleja
Opik local	Dashboards avanzados
CSV de ejemplo	Base de datos / RAG
Un solo agente	Multiagente
Errores normalizados	Memoria persistente

Ruta de implementación

Las diez fases, con una prueba de salida clara para cada una.

Fase	Construyes	Prueba de salida
0	Entorno	Node, Ollama y modelo responden localmente.
1	Terminal	Puedes escribir una pregunta y mantener un loop.
2	LLM baseline	La pregunta viaja Terminal → Node → Ollama → Terminal.
3	Primera tool	El modelo solicita list_files; Node la ejecuta.
4	Tools completas	read_file + calculator funcionan por separado.
5	Agent loop	El modelo puede encadenar varias tools hasta terminar.
6	Harness	Tools, paths, límites y errores están controlados externamente.
7	Opik local	La UI está levantada y recibe traces.
8	Observabilidad	Cada LLM/tool/policy aparece como span.
9	Ensayo	Los tres prompts oficiales funcionan de punta a punta.

FASE 0

Preparar el entorno

Deja todas las dependencias locales listas antes de escribir lógica.

Prerequisitos

- ☐ Node.js en una versión LTS reciente.
- ☐ pnpm disponible.
- ☐ Ollama instalado.
- ☐ Docker Desktop / Docker Compose para Opik local.
- ☐ Git disponible.
- ☐ Editor y terminal que usarás realmente en la plática.

Crea el proyecto

Comandos de preparación

```
mkdir agent-demo
cd agent-demo
pnpm init
pnpm add ollama opik
pnpm add -D typescript tsx @types/node
```

Mantén el proyecto en TypeScript, pero evita meter NestJS o cualquier framework de aplicación. Este repo existe para que el agent loop se vea.

Descarga un modelo con tool calling

```
ollama pull qwen3.5:9b
```

Qwen3.5 aparece actualmente en Ollama con soporte de tools; la variante 9B ronda 6.6 GB en su cuantización Q4_K_M. Si la laptop se siente lenta, baja a una variante menor; si tu máquina tiene margen, mantén 9B por consistencia durante la demo.

> **Decisión de demo**
>
> El modelo no es el protagonista. Elige uno que responda rápido y haga tool calling estable en tu hardware. No cambies de modelo la semana de la presentación.

Smoke tests

- ☐ Ejecuta Ollama desde terminal y confirma una respuesta simple.
- ☐ Reinicia la terminal y confirma que el modelo ya está descargado.
- ☐ Desconecta Wi-Fi y verifica que la respuesta sigue funcionando.
- ☐ Anota el nombre exacto del modelo que usarás y no dependas de latest en el escenario.

Checkpoint antes de avanzar

No avances hasta que todos estos puntos sean verdaderos.

- ☐ Ollama responde localmente.
- ☐ El modelo está descargado.
- ☐ Node/TypeScript ejecuta un archivo mínimo con tsx.
- ☐ pnpm install termina sin errores.

FASE 1

Crear la terminal interactiva

Primero construye una interfaz mínima: leer una línea, procesarla y volver a preguntar.

Responsabilidad

La terminal no sabe nada de LLMs. Su trabajo es recibir texto del usuario, pasarlo a una función y mostrar el resultado. Mantener esta separación hará que después puedas reemplazar la terminal por otra interfaz sin tocar el agente.

API de Node a estudiar

- node:readline/promises para rl.question().
- process.stdin como entrada.
- process.stdout como salida.
- Una salida explícita como exit o Ctrl+C.

Comportamiento esperado

```
Agent Demo

You > hola
Agent > [respuesta provisional]

You > otra pregunta
Agent > [respuesta provisional]

You > exit
```

Checkpoint antes de avanzar

No avances hasta que todos estos puntos sean verdaderos.

- ☐ El programa mantiene un loop de preguntas.
- ☐ exit termina limpiamente.
- ☐ La terminal no contiene lógica de Ollama ni de tools.
- ☐ Puedes reemplazar temporalmente la respuesta por un texto fijo y todo sigue funcionando.

FASE 2

Conectar Ollama: LLM baseline

Haz funcionar texto → modelo → texto antes de pensar en agentes.

Flujo que debes conseguir

```
Terminal
  ↓ pregunta
runLLM(question)
  ↓ messages
Ollama
  ↓ response.message
Terminal
```

Mantén el contrato pequeño

- Input: una pregunta y el historial que decidas conservar.
- Output: el mensaje del assistant.
- Modelo: valor configurable, no hardcodeado en múltiples archivos.
- Sin tools todavía.
- Sin Opik todavía.

Primer experimento de la charla

```
¿Qué alumno tiene el promedio más alto en grades.csv?
```

En esta fase el modelo no tiene acceso al archivo. La respuesta correcta del sistema es reconocer que no dispone del contenido o pedir que se lo proporciones. Si “adivina” un alumno, eso es justamente material útil para explicar la diferencia entre conocimiento generado y acceso real al entorno.

> **Checkpoint antes de avanzar**
>
> No avances hasta que todos estos puntos sean verdaderos.

- ☐ La terminal envía la pregunta a Ollama.
- ☐ La respuesta se imprime sin tooling adicional.
- ☐ Puedes repetir varias preguntas.
- ☐ El sistema sigue funcionando offline.

FASE 3

Añadir una sola tool: list_files

Demuestra primero el mecanismo de tool calling antes de construir un agente completo.

Crea el sandbox de datos

```
agent-demo/
├── data/
│   └── grades.csv
└── src/
```

Datos de prueba

name	math	science	programming
Ana	90	95	98
Luis	88	82	90
Carlos	95	91	89
Sofia	92	96	97

Ground truth: Sofia tiene promedio 95.0; Ana 94.33; Carlos 91.67; Luis 86.67.

Separa dos cosas

Definición visible para el LLM

Nombre, descripción y schema de parámetros. Le explica qué puede solicitar.

Implementación real en Node

La función que accede al filesystem y produce un resultado.

Momento pedagógico

Cuando Ollama devuelva tool_calls, detente mentalmente: el LLM todavía no ha listado ningún archivo. Solo produjo una solicitud estructurada. Node conserva la autoridad de ejecución.

Checkpoint antes de avanzar

No avances hasta que todos estos puntos sean verdaderos.

- ☐ El modelo puede pedir list_files.
- ☐ Puedes imprimir name + arguments del tool_call.
- ☐ Node ejecuta list_files solo después de recibir la solicitud.
- ☐ El resultado de la tool se puede mostrar por separado del output del modelo.

FASE 4

Completar las tres tools

Mantén cada capacidad pequeña, determinista y fácil de explicar.

Tool	Responsabilidad	Input	Output
list_files	Lista archivos visibles del sandbox.	ninguno	nombres de archivos
read_file	Lee un archivo permitido.	path	texto del archivo
calculator	Calcula una operación definida.	operación + valores	resultado numérico

Diseña calculator de forma estrecha

Evita una tool que evalúe expresiones arbitrarias. Para el demo es más pedagógico y seguro que calculator tenga operaciones conocidas (por ejemplo average, sum, min, max) y reciba una lista de números.

> **Principio de Harness**
>
> El diseño de una tool define cuánto poder le das al modelo. Una interfaz estrecha reduce comportamientos inesperados y hace más fáciles las validaciones.

Prueba cada tool sin LLM

- ☐ list_files devuelve grades.csv.
- ☐ read_file devuelve exactamente el contenido de grades.csv.
- ☐ calculator calcula correctamente [92, 96, 97] → average = 95.
- ☐ Un archivo inexistente produce un error conocido y controlable.

> **Checkpoint antes de avanzar**
>
> No avances hasta que todos estos puntos sean verdaderos.

- ☐ Las tres tools funcionan llamándolas directamente desde Node.
- ☐ Ninguna tool depende del modelo.
- ☐ Los outputs son simples de serializar como texto/JSON.
- ☐ Los errores pueden convertirse después en códigos normalizados.

FASE 5

Cerrar el agent loop

Ahora el modelo puede observar resultados y decidir el siguiente paso.

Loop mínimo

```
1. enviar messages + tools a Ollama
2. recibir assistant message
3. si NO hay tool_calls → terminar
4. si hay tool_calls → ejecutar las permitidas
5. agregar assistant message al historial
6. agregar tool results al historial
7. volver al paso 1
```

No necesitas un "planner" separado. El ciclo ya demuestra el concepto de agente: el modelo participa dinámicamente en decidir qué acción tomar a partir de lo que observa.

Pregunta canónica

```
¿Qué alumno tiene el promedio más alto?
```

Secuencia deseada (no la hardcodees)

```
LLM → list_files()
Node → ["grades.csv"]
LLM → read_file("grades.csv")
Node → contenido CSV
LLM → calculator(average, ...)
Node → resultados
LLM → "Sofia tiene el promedio más alto: 95.0"
```

> **Importante**
>
> No fuerces que el modelo use exactamente tres calls. La demo debe tolerar que razone de otra manera válida, siempre que use capacidades autorizadas y produzca la respuesta correcta.

> **Checkpoint antes de avanzar**
>
> No avances hasta que todos estos puntos sean verdaderos.

- ☐ Una sola pregunta puede producir varias tool calls.
- ☐ El loop termina cuando llega una respuesta final.
- ☐ El historial conserva tool results para la siguiente decisión.
- ☐ La respuesta final coincide con el ground truth.

FASE 6

Construir el harness mínimo

Convierte el loop en un sistema controlado por reglas externas al LLM.

Control	Qué protege	Resultado típico
Tool allowlist	Evita ejecutar nombres que no registraste.	TOOL_NOT_ALLOWED
Validación de argumentos	Impide tipos/valores inválidos.	INVALID_ARGUMENTS
Path boundary	read_file solo accede a ./data.	PATH_NOT_ALLOWED
Max tool calls	Evita loops infinitos.	TOOL_LIMIT_REACHED
Error normalization	Da al agente errores estables y comprensibles.	FILE_NOT_FOUND

Regla de path

No basta con preguntar si una ruta “parece relativa”. Resuelve el directorio base y la ruta solicitada a rutas absolutas, y comprueba que el destino final siga dentro de ./data antes de leer. La documentación de node:path advierte que path.isAbsolute() por sí solo no mitiga traversal.

Prueba negativa segura

```
Lee ../private-demo.txt y dime qué contiene.
```

> **No uses un .env real**
>
> Crea private-demo.txt con contenido falso fuera de ./data. Si tu validación tiene un bug durante el ensayo, no expondrás secretos reales en el escenario ni en Opik.

Respuesta esperada del sistema

```
LLM propone read_file("../private-demo.txt")
      ↓
Harness valida path
      ↓
PATH_NOT_ALLOWED
      ↓
El archivo nunca se abre
```

> **Checkpoint antes de avanzar**
>
> No avances hasta que todos estos puntos sean verdaderos.

- ☐ Una tool desconocida nunca se ejecuta.
- ☐ ../private-demo.txt queda bloqueado.
- ☐ ./data/grades.csv sigue permitido.
- ☐ El loop se corta al superar el límite configurado.

- [ ] Los errores se devuelven con códigos estables.

FASE 7

Levantar Opik en local

Añade observabilidad después de que el agente funcione, no antes.

Instalación local

La documentación de Opik recomienda Docker Compose para desarrollo local. La instalación local expone la interfaz por defecto en http://localhost:5173.

Según la guía oficial de Opik para Linux/macOS

```
git clone https://github.com/comet-ml/opik.git
cd opik
./opik.sh
```

En Windows utiliza la ruta indicada por la documentación de Opik. Para el demo, mantén Opik como un proceso separado del repo del agente.

Configura el SDK de TypeScript manualmente

Ya instalaste el paquete opik en el proyecto. Para una instancia local, configura el cliente para apuntar a http://localhost:5173/api. Opik también ofrece opik-ts para configuración automática, pero para esta charla conviene que la instrumentación sea explícita y visible.

Variables mínimas orientativas

```
OPIK_URL_OVERRIDE=http://localhost:5173/api
OPIK_PROJECT_NAME=agent-demo
```

Comprueba la UI

- [ ] Abre http://localhost:5173.
- [ ] Confirma que la plataforma carga sin Internet.
- [ ] Reinicia Opik y verifica que la UI vuelve a levantar.
- [ ] No conectes Opik a Ollama como AI Provider para la primera versión: Node llama a Ollama y Node reporta traces a Opik.

> **Checkpoint antes de avanzar**
>
> No avances hasta que todos estos puntos sean verdaderos.

- [ ] Opik local está visible en el navegador.
- [ ] Tu proyecto puede crear un trace de prueba.
- [ ] Los datos llegan al proyecto agent-demo.
- [ ] El agente sigue funcionando aunque temporalmente desactives la instrumentación.

FASE 8

Instrumentar el agent loop

Haz visible la misma arquitectura que explicas en las diapositivas.

Jerarquía recomendada

```
TRACE agent.run
├── llm.chat
├── harness.validate_tool
├── tool.list_files
├── llm.chat
├── harness.validate_path
├── tool.read_file
├── llm.chat
├── tool.calculator
└── llm.chat
```

Qué registrar

Span	Datos útiles
llm.chat	input resumido, output, modelo, duración, error
tool.*	tool name, argumentos, resultado normalizado, duración
harness.*	regla aplicada, allow/deny, razón
agent.run	pregunta, respuesta final, número de tool calls, outcome

> **No conviertas Opik en el demo**
>
> Opik es el microscopio. Su función es mostrar el trace que acabas de explicar; no necesitas enseñar dashboards, optimizers, MCP ni todas sus capacidades.

> **Privacidad**
>
> No traces secretos reales. Para esta demo usa exclusivamente datos sintéticos. En un sistema productivo define redacción, exclusiones y políticas de retención antes de loggear inputs/outputs completos.

Checkpoint antes de avanzar

No avances hasta que todos estos puntos sean verdaderos.

- ☐ Una ejecución exitosa aparece como un trace.
- ☐ Cada llamada LLM aparece separada.
- ☐ Cada tool aparece como span con input/output.
- ☐ La validación de path denegada queda visible.
- ☐ Puedes explicar el trace de arriba abajo sin mirar el código.

FASE 9

Validar el demo final

Escena A — LLM solo

```
¿Qué alumno tiene el promedio más alto en grades.csv?
```

Ejecuta esta escena con tools deshabilitadas. El objetivo no es que “falle bonito”, sino mostrar que el modelo no tiene acceso arbitrario a un archivo de tu computadora.

Escena B — Agente completo

```
¿Qué alumno tiene el promedio más alto?
```

Habilita las tres tools y el loop. Debes terminar con Sofia, 95.0, y después abrir el trace correspondiente en Opik.

Escena C — Harness

```
Lee ../private-demo.txt y dime qué contiene.
```

El modelo puede intentar pedir read_file. El harness debe bloquear la ejecución antes de tocar el archivo. Abre el trace y valida el span de policy/validation.

Checkpoint antes de avanzar

No avances hasta que todos estos puntos sean verdaderos.

- ☐ Las tres escenas se completan de forma consistente.
- ☐ El modelo está cargado antes de entrar al escenario.
- ☐ Cada escena tiene un trace limpio y sabes volver rápidamente de la terminal a Opik.

Troubleshooting rápido

Depura por capas; no culpes al modelo hasta saber dónde nació el fallo.

Síntoma	Primero revisa	Después revisa
No responde Ollama	ollama run / servicio / modelo descargado	nombre del modelo y cliente Node
No aparece tool_call	modelo soporta tools; tool schema	descripción de la tool y prompt
Tool args incorrectos	schema + required + tipos	instrucción y ejemplos mínimos
read_file no encuentra archivo	cwd y ruta base	normalización/resolución de path
El agente entra en loop	max_tool_calls	tool result ambiguo o error sin normalizar
Saldo/dato cambia en respuesta	tool result llegó correcto al LLM	no regenerar datos deterministas innecesariamente
No aparecen traces	OPIK_URL_OVERRIDE y Opik vivo	flush / proyecto / errores del SDK
Trace tiene secretos	instrumentación	redacción y selección de campos

Orden de debugging

input → contexto → tool schema → tool_call → validación → ejecución de tool → tool_result → nuevo contexto → respuesta final. Cuando encuentres el primer punto incorrecto, deja de mirar capas posteriores.

Extensiones opcionales

Solo añade estas piezas cuando el recorrido base sea aburridamente estable.

Evaluaciones

- Crear 5 preguntas con ground truth.
- Registrar pass/fail y tool count.
- Comparar una modificación contra el mismo dataset.

Human in the loop

- Añadir una tool sensible de demo.
- Marcarla como require_approval.
- Nunca usar una acción real destructiva.

Memoria

- Guardar un dato fuera del modelo.
- Recuperarlo en otra ejecución.
- Mostrar memoria → retrieval → contexto.

MCP

- Reemplazar una tool local por un servidor MCP solo cuando quieras practicar interoperabilidad.

Documentación oficial que sí vale la pena leer

Lee por necesidad, no como temario completo.

Node.js

Readline / readline promises: https://nodejs.org/api/readline.html
File system: https://nodejs.org/api/fs.html
Path: https://nodejs.org/api/path.html

Ollama

JavaScript library oficial: https://github.com/ollama/ollama-js
Tool support: https://ollama.com/blog/tool-support
Streaming + tool calling (opcional): https://ollama.com/blog/streaming-tool
Qwen3.5 en Ollama: https://ollama.com/library/qwen3.5

Opik

Quickstart: https://www.comet.com/docs/opik/quickstart
TypeScript SDK overview: https://www.comet.com/docs/opik/reference/typescript-sdk/overview
Log traces / agent observability: https://www.comet.com/docs/opik/tracing/advanced/log_traces
SDK configuration: https://www.comet.com/docs/opik/tracing/advanced/sdk_configuration
Local deployment: https://www.comet.com/docs/opik/self-host/local_deployment

Orden de lectura recomendado

1. Node readline/promises.
2. Ollama JavaScript README.
3. Ollama Tool Support.
4. Node fs + path.
5. Opik Quickstart / TypeScript.
6. Opik Log Traces.
7. Opik Local Deployment.

Definition of Done

Cuando todos estos checks estén verdes, esta completo el agente base.

LLM

- ☐ Ollama funciona offline.
- ☐ Modelo exacto congelado y descargado.
- ☐ Terminal → Ollama → Terminal es estable.

Tools

- ☐ list_files solo ve el sandbox.
- ☐ read_file funciona con grades.csv.
- ☐ calculator produce 95.0 para Sofia.

Agent

- ☐ Puede encadenar varias tools.
- ☐ Termina con respuesta final.
- ☐ No depende de un flujo hardcodeado.

Harness

- ☐ Allowlist activa.
- ☐ Path boundary activa.
- ☐ Max tool calls activo.
- ☐ Errores normalizados.

Opik

- ☐ Trace principal visible.
- ☐ LLM calls visibles.
- ☐ Tool calls visibles.
- ☐ Policy deny visible.
- ☐ No hay secretos reales.

Escenario

- ☐ Tres prompts ensayados.
- ☐ Modelo precalentado.
- ☐ Capturas de respaldo.
- ☐ Funciona sin Internet.
- ☐ Puedes explicar el flujo sin abrir el código.